

ReplayCapsule-RV: Event-Sufficient Hardware Failure Capsules for Replaying Embedded RISC-V Interrupt/MMIO Bugs

Anonymous Author(s)

ABSTRACT

Embedded firmware failures often depend on interrupt timing and memory-mapped I/O values that are visible to software but absent from ordinary crash reports. ReplayCapsule-RV records commit-indexed capsules for a scoped class of single-hart RV32I interrupt/MMIO bugs and replays those boundary events in an RTL harness. The implementation integrates a record-side hardware capsule path with PicoRV32, compiler-built RV32I firmware images, Verilator record/replay execution, corruption checking, runtime summaries, and same-target FPGA mapping. The generated artifact reports 45/45 compiler-backed full RTL replay PASS rows and rejects 10 replay-critical corrupted capsules with 0 unexpected accepts. The current prototype validates event sufficiency and replay reproducibility for bring-up/debug instrumentation; it does not claim an autonomous full-core hardware replay engine or area-optimized shipped silicon.

1 INTRODUCTION

Embedded firmware often fails because the external world arrives at the wrong architectural moment. A sensor value crosses a threshold, an interrupt lands inside a short critical section, or a command register is sampled before firmware has restored an invariant. These failures are difficult to reproduce after the fact because the decisive facts are boundary events: interrupt delivery and return, MMIO values, and property context. Full instruction or processor-trace mechanisms can preserve far richer execution histories, but many firmware validation failures need a narrower replay object: the events that drive the same firmware-visible path to the same violation.

ReplayCapsule-RV studies that point in the design space. For a deterministic single-hart RV32I platform with the same firmware image and initial architectural state, the recorder captures commit-indexed interrupt/MMIO events, selected boundary observations, and property-failure metadata. The replay harness consumes those events at the same commit boundary and checks property metadata, event payloads, and final signatures. The design is deliberately scoped: it records replay-driving facts for a defined class of embedded failures rather than attempting to replace full trace, snapshots, or platform-wide debug infrastructure.

This paper makes three contributions:

- a commit-indexed boundary-event model for replaying scoped single-hart RV32I interrupt/MMIO failures;
- a record-side ReplayCapsule-RV hardware path and host-driven RTL replay harness that validate those events against compiler-built firmware;
- a generated evidence package covering replay success, corruption rejection, runtime cost, buffer-depth tradeoffs, and same-target FPGA overhead.

The evaluation is intentionally audit-oriented. The artifact reports compiler-backed firmware builds, 45/45 full RTL replay PASS

Deterministic Replay Methodology

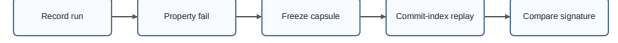


Figure 1: Record/replay timeline with commit-indexed events. Source generated from scripts/make_figures.py.

rows, full RTL corrupted-capsule checks, runtime overhead summaries across baseline and recorder configurations, and full-core ECP5 mapped results. Claims outside this scope are treated as limitations rather than inferred results.

2 MOTIVATION

The motivating failures are firmware bugs whose manifestation depends on external ordering rather than arithmetic alone. The benchmark suite covers sensor threshold handling, interrupt races, MMIO ordering, stack protection, UART command handling, and watchdog timeout behavior. Each family has compiler-built failing and fixed firmware images; selected families also include a no-failure edge case.

The threat model is a debugging and validation setting, not an attacker model. The recorder may observe firmware-visible events and property-checker outputs. Replay starts from the same image and initial architectural state. Unsupported nondeterminism is out of scope unless it is modeled as an event. In particular, multicore interleavings, DMA, caches/coherence effects, analog device state, and undefined C behavior are not covered by the current artifact.

3 EVENT-SUFFICIENT REPLAY MODEL

The replay model is defined over architectural transitions. The original run and replay run share the same platform profile, firmware image, and initial architectural state. Between boundary events, the RV32I core semantics are deterministic. At a boundary, replay consumes the recorded event at the same commit index and validates replay-visible observations.

An event contains a commit index, a same-index order tag, a class, and a payload. Core classes are those required to drive replay in the scoped failures: MMIO read values, MMIO write observations, interrupt entry and exit, external inputs, selected stores or branch outcomes when they influence the property, checkpoint/hash values when used, and the property-failure signature. Diagnostic PC context is useful for debugging and comparator alignment, but the model does not assert a globally smallest trace.

Figure 1 shows the intended record/replay timeline. The recorder captures only the boundary facts needed to make the replay run take the same firmware-visible path through the target failure prefix. Overflow invalidates replay availability; it is a status bit, not a recoverable compression mode.

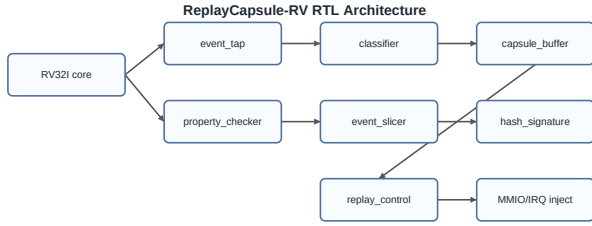


Figure 2: ReplayCapsule hardware architecture. Source generated from scripts/make_figures.py.

The proof sketch in `formal/replay_sufficiency_theorem.md` is by induction over commit index: equal initial state, deterministic ordinary steps, equal boundary-event payloads at nondeterministic steps, and deterministic property checking over the equal prefix.

4 HARDWARE ARCHITECTURE

ReplayCapsule-RV instruments the processor boundary rather than replacing the core. The PicoRV32 wrapper exposes memory, MMIO, interrupt, and commit-context signals to the record-side modules. The event tap observes candidate facts, the classifier selects replay-relevant and diagnostic classes, the slicer retains context around a property failure, the capsule buffer stores encoded packets, and the property checker freezes the capsule on a target violation.

The board-level mapped wrappers expose only clock, reset, UART, GPIO, status LEDs, and three recorder-status pins. Large debug and internal buses remain inside the FPGA design. A v1 mapped build disables redundant v2 recorder instances that are not selected by the architecture parameter, but the remaining recorder, event packet storage, signature path, property checker, and status visibility still cost substantial resources. The current same-target ECP5 rows report LUT overhead from 114.20% to 221.23%, FF overhead from 174.53% to 646.43%, and Fmax change from -19.56% to -9.98%. This overhead is acceptable for the intended bring-up/debug-instrumentation use case, where fidelity and auditability matter more than shipped-silicon area; it is not presented as an optimized always-on production block.

4.1 ReplayCapsule-RV v2 Prototype

ReplayCapsule-RV v2 keeps the v1 recorder intact and adds a separately selectable compressed recorder prototype. The v2 path packs replay-critical event class, commit-delta, address-token, payload-hash, and limited debug context into 64-bit capsule words; repeated MMIO addresses can use a small dictionary, and diagnostic context can be dropped under adaptive buffer pressure while replay-critical MMIO, interrupt, property-fail, and checkpoint-hash events remain capturable. The v2 implementation also includes a synthesizable replay-consume controller prototype that validates event type, commit order, MMIO value, interrupt cause, and payload hash before reporting that all capsule events were consumed.

The v2 recorder and replay consumer are not presented as a replacement for the measured v1 full RTL replay flow. In this artifact, v2 recorder RTL is selectable in the full-core host-driven record

path, and the replay-consume controller has directed RTL corruption tests plus standalone ECP5 prototype mapping. The replay consumer is not wired to full-core instruction replay: it has a capsule stream input and observed-event comparison inputs, but no integrated capsule memory source, core MMIO/IRQ replay mux, or autonomous replay-mode controller in the PicoRV32 full-core flow.

5 RECORD/REPLAY IMPLEMENTATION

The replay comparator consumes an expected capsule and an observed replay trace. A passing comparison requires matching property metadata, the required event classes at the selected commit or cycle index, exact payload equality for replay-driving events, and no extra replay-driving events. Negative tests corrupt metadata, remove events, duplicate events, shift indices, swap order tags, and perturb payloads. The same comparator is used for model fixtures and exported RTL-smoke capsules.

The current replay-consume path is host-driven. The Verilator harness records a capsule from the instrumented PicoRV32 wrapper, reruns the same compiler-backed firmware under replay control, injects the recorded interrupt/MMIO boundary facts, and checks the resulting trace. This is strong evidence for the event stream and RTL integration, but it is not a synthesizable replay-consume datapath.

Bounded local checks cover recorder and control contracts such as event classification, buffer behavior, property signatures, replay mismatch handling, register access, hash/signature update, and MMIO/interrupt logging. These checks support local RTL obligations; they are not a mechanized end-to-end processor theorem.

6 EVALUATION

The evaluation is generated from repository scripts and processed CSVs; the paper-number audit rejects numeric claims that do not appear in generated evidence. The core full-RTL ledger reports 45/45 compiler-backed record/replay PASS rows. The negative ledger rejects 10 replay-critical corruptions, reports 0 unexpected accepts, and leaves 2 not-applicable rows for corruption classes outside the current commit-index harness.

6.1 Replay, Scale, and Buffers

Replay succeeds on the fixed compiler-backed suite in the full-RTL harness, but scaling is constrained by capsule capacity. The regenerated workload-scaling evidence reports 375/375 PASS rows with workload-aware v1 depths: 256 for smoke and short workloads, 1024 for medium, 4096 for long, and 16384 for stress. The buffer-sensitivity table shows replay success reaches 100.000000% for each workload class at those selected depths, while the sticky overflow bit remains asserted on long-running fixed and no-failure rows that do not freeze at a property failure. The paper therefore treats buffer depth as a design parameter, not a hidden constant.

Capsule-size comparisons use the buffer depth required for the workload class under test. `capsule_baseline_summary.csv` reports median ReplayCapsule reductions versus the raw full-instruction-trace estimate of 59.52% for smoke, 78.71% for short, 83.18% for medium, 64.86% for long, and 22.84% for stress. The same CSV includes a labeled `riscv_etrace_branch_trace_estimate` row;

that estimate is smaller than ReplayCapsule-RV for these workloads, so the raw instruction-trace comparison is a weak baseline, not a state-of-the-art trace-compression claim.

The v2 evidence package adds compressed recorder configurations and two additional actuator/configuration firmware families. Those rows improve workload and benchmark coverage for the scoped model, but they do not turn the standalone replay-consume controller into an integrated full-core autonomous replay path.

6.2 Cost and Mapping

The mapped FPGA results are same-target bring-up/debug measurements. The board-level baseline and ReplayCapsule wrappers use the same ecp5-85k target and yosys+synth_ecp5+nextpnr-ecp5 flow. After gating unselected v2 recorder instances in v1 mapped builds, the representative mapped-scaling summary reports LUT overhead from 114.20% to 221.23%, FF overhead from 174.53% to 646.43%, BRAM overhead of 0.00%, and Fmax change from -19.56% to -9.98%. Expanded mapped-scaling CSVs report the lower-overhead core configurations separately from the full diagnostic configuration.

This overhead is acceptable only under the intended use case: temporary validation and post-failure replay instrumentation during hardware/software bring-up. It is not presented as a production always-on block. The current artifact does not include an ASIC or power flow; FPGA results should be read as feasibility and cost-shape evidence rather than signoff implementation data.

7 RELATED WORK

RISC-V E-Trace, N-Trace, and external debug specifications target processor observability, control-flow reconstruction, and standardized debug transport [6, 7]. ReplayCapsule-RV is complementary: it records commit-indexed replay-driving interrupt/MMIO events for a scoped single-hart RV32I failure class. It does not attempt to provide architectural trace coverage or debug interoperability.

Post-silicon trace compression reduces trace bandwidth and storage by exploiting structure in instruction or control-flow streams. ReplayCapsule-RV makes a different tradeoff. It records the boundary events needed by the replay harness, not a compressed representation of the full executed instruction stream.

Deterministic replay systems record nondeterministic inputs and ordering so an execution can be reconstructed [1, 4]. ReplayCapsule-RV borrows that replay discipline but narrows the target to embedded interrupt/MMIO failures, uses commit-indexed hardware capsules, and evaluates a PicoRV32 RTL harness rather than an operating-system or virtual-machine replay substrate.

Runtime verification and assertion monitors detect violations close to the hardware/software boundary [5]. ReplayCapsule-RV uses property checking as the capsule trigger, but its contribution is the replay-driving event stream and the evidence that corrupted capsules fail comparison.

Embedded event logs are widely used to explain firmware failures after deployment. ReplayCapsule-RV differs by making event ordering architectural through commit indices and by validating the capsule through replay comparison instead of presenting the log only as diagnostic text.

FPGA/emulation debugging and checkpointing can capture broad hardware state and rerun workloads under controlled conditions. Snapshot and time-travel systems similarly preserve enough state to resume or inspect execution [3]. ReplayCapsule-RV is narrower: it keeps the same initial state and firmware image, then records the boundary events needed for a selected failure prefix.

The side-by-side distinction is:

- Full instruction or commit traces capture the retired stream or per-commit state; ReplayCapsule-RV records replay-driving boundary events and property-fail context.
- RISC-V E-Trace and N-Trace-style mechanisms capture compressed control-flow discontinuities and synchronization; the generated E-Trace-style estimate is a stronger size baseline than raw instruction trace, while ReplayCapsule-RV carries MMIO values and interrupt timing for replay.
- Deterministic VM replay captures device inputs and scheduling at OS or VM granularity; ReplayCapsule-RV uses commit-indexed single-hart RV32I boundary events inside a firmware-running RTL harness.
- Runtime-verification monitors capture assertion outcomes and checker state; ReplayCapsule-RV uses the checker as a trigger and validates the captured event stream by record/replay comparison.

In the current implementation, v1 packets are 168.000000-bit boundary-event records and v2 measured rows use compact 64-bit exported capsule words where integrated; the claim is scoped replay sufficiency for interrupt/MMIO bugs, not trace-standard replacement or autonomous full-core replay consume.

The implementation builds on PicoRV32, Verilator, Icarus Verilog, Yosys, nextpnr, and bounded formal tooling [2, 8–12]. These tools support reproducibility; the claimed research object is the scoped capsule model, recorder, replay harness, and evidence package.

8 LIMITATIONS

The scope is single-hart RV32I firmware with deterministic core semantics, the same program image, the same initial architectural state, and modeled external events. The current theorem excludes multicore interleavings, DMA unless modeled, cache/coherence effects unless modeled, analog nondeterminism, and claims about undefined C behavior.

MMIO side effects are modeled only at the replay-visible boundary. Interrupts are injected at an architectural boundary. Replay consumes every required event exactly once; missing, duplicate, reordered, or corrupted events must fail comparison. Capsule overflow invalidates replay availability for failure reconstruction unless the replay-critical prefix is still complete.

The replay-consume path is host-driven in the Verilator harness. The mapped design measures the recorder/instrumented full-core wrapper, not a synthesizable autonomous replay-consume engine. The v1 workload evidence recovers 375/375 replay PASS rows only by using workload-aware depths: 256 for smoke and short, 1024 for medium, 4096 for long, and 16384 for stress. These depths increase capsule bytes and reduce the median stress-workload reduction versus a raw full-instruction trace to 22.84%. High FF/LUT overhead remains an optimization target: after gating unselected v2

recorders in v1 mapped builds, same-target ECP5 overhead is still as high as 221.23% LUT and 646.43% FF. ASIC power/area/timing is not evaluated, and no generic-cell or open-PDK result should be inferred from the FPGA flow. The evaluation is single-hart RV32I and does not cover multicore, DMA, caches/coherence, or arbitrary peripherals.

ReplayCapsule-RV v2 reduces the architectural risk by adding compressed recorder RTL and a synthesizable replay-consume controller prototype, but the prototype is not fully integrated into the full-core replay harness. The consumer has tested stream consumption, order checking, MMIO/interrupt checking, and payload-hash rejection, and it maps as a standalone ECP5 block. It is still missing the full-core replay datapath: capsule storage/streaming in the full-core top, core MMIO/IRQ replay muxing, observed-event plumbing from PicoRV32, and an autonomous replay-mode controller. These rows should not be used to claim autonomous full-core hardware replay.

9 CONCLUSION

ReplayCapsule-RV shows that a scoped class of embedded RV32I interrupt/MMIO failures can be recorded as commit-indexed boundary-event capsules and replayed in a host-driven RTL harness with compiler-backed firmware. The artifact reports 45/45 full RTL replay PASS rows, corruption rejection, runtime summaries, buffer-depth tradeoffs, and same-target FPGA overhead. The result is not a replacement for trace standards or snapshots; it is a record-side hardware capsule design and evidence package for replay-driving boundary events. The next step is an integrated full-core replay-consume datapath and broader platform modeling, not stronger claims from the current prototype.

REFERENCES

- [1] George W. Dunlap, Samuel T. King, Sukru Cinar, Murtaza A. Basrai, and Peter M. Chen. 2002. ReVirt: Enabling Intrusion Analysis through Virtual-Machine Logging and Replay. In *OSDI*.
- [2] Icarus Verilog Project. 2026. Icarus Verilog. <https://steveicarus.github.io/iverilog/>.
- [3] Samuel T. King, George W. Dunlap, and Peter M. Chen. 2005. Debugging Operating Systems with Time-Traveling Virtual Machines. In *USENIX Annual Technical Conference*.
- [4] Thomas J. LeBlanc and John M. Mellor-Crummey. 1987. Debugging Parallel Programs with Instant Replay. In *IEEE Transactions on Computers*.
- [5] Martin Leucker and Christian Schallhart (Eds.). 2009. *Runtime Verification*. Springer.
- [6] RISC-V International. 2026. RISC-V External Debug Support. <https://github.com/riscv/riscv-debug-spec>. Accessed 2026-06-26.
- [7] RISC-V International. 2026. RISC-V Processor Trace Specification. <https://github.com/riscv-non-isa/riscv-trace-spec>. Accessed 2026-06-26.
- [8] Verilator Project. 2026. Verilator. <https://verilator.org/>.
- [9] Clifford Wolf. 2026. PicoRV32: A Size-Optimized RISC-V CPU. <https://github.com/YosysHQ/picorv32>.
- [10] YosysHQ. 2026. nextpnr: portable FPGA place and route. <https://github.com/YosysHQ/nextpnr>.
- [11] YosysHQ. 2026. SymbiYosys and Yosys SMTBMC. <https://yosyshq.readthedocs.io/projects/sby/>.
- [12] YosysHQ. 2026. Yosys Open SYnthesis Suite. <https://yosyshq.net/yosys/>.